Emission-Factor Version Explainer — Build Playbook

A free-data, no-license MVP that proves the "explain-the-delta" wedge — and a step-by-step guide to building it with Claude Code even though you're not a developer.

Owner: Suhas Pala · Version: 1.0 · Stack cost: €0

PART 0 — Executive summary (plain English, no jargon)

The problem, in one sentence. Every year the official databases that tell companies "one litre of diesel = X kg of CO_2" quietly change their numbers. When they do, every company's carbon footprint silently shifts — and someone (usually an expensive consultant like you were) has to figure out *what changed, why, and whether it breaks the company's climate targets.* Today that work is done by hand, in Excel and Word.

What we're building. A small piece of software that:

1. Reads last year's and this year's official emission-factor tables (free UK government data).
2. Spots every number that moved meaningfully.
3. Takes a real product's footprint and recalculates it with the new numbers.
4. Uses AI to **write a clear, defensible explanation** of what changed, why, and whether it threatens the company's targets — the report a consultant would otherwise spend days writing.

Why it matters. The big platforms (CO2 AI, Watershed) *recompute* the new number for you. None of them *explain* it in a client-ready, methodology-sound way — that's still human work, and it's exactly the expert judgment you spent four years doing. This tool packages your judgment.

Why you specifically. You did emission-factor matching, version reconciliation, and methodology-report writing by hand *repeatedly* at Quantis (furniture EF matching, the eQopsphere harmonization, the dairy year-over-year trend reports). You know what "wrong" and "why did this change" look like. A generic AI engineer does not.

What "done" looks like for the MVP. You type in (or upload) a product, click a button, and get back a one-page report: "Your footprint went from 12.4 to 13.1 kg CO_2e (+5.6%). The main driver is the UK grid electricity factor rising 8%, because the 2026 update reflects [reason]. This would push you above your flat 2025 SBTi baseline — flag for review." Built on free data, no ecoinvent licence, no client secrets.

What this is FOR (be honest with yourself).

- **Primary goal:** a credibility artifact that gets you hired at carbmee, Planet A, Makersite, Henkel, Footprint Intelligence, etc. — the exact roles you're applying to.
- **Secondary goal:** a wedge product for consultancies / SMEs who can't afford the big platforms.
- **Not the goal:** beating CO2 AI. Don't try.

How long. A focused person doing evenings can reach a demo in ~2–3 weeks using Claude Code, which writes most of the actual code. Your job is direction, domain judgment, and testing — not typing Python.

PART 1 — Your competitive position (keep this honest)

Layer	Who owns it	Should you compete?
Footprint generation at scale	CO2 AI, Watershed, Makersite	**No.** Built, funded, enterprise-priced.
Emission-factor API / versioned data	Climatiq, Carbonfact	No — use them later, don't rebuild.
Explaining a version delta for a specific model	**Nobody (done by hand)**	**YES — this is the wedge.**
Validation / methodology narrative / audit	**Consultants (human)**	**YES — your moat.**
Regulatory conformance (CSRD/DPP/SBTi)	Partial, fragmented	Later — as a feature of the above.

Evidence the wedge is open: Carbonfact explains ecoinvent updates via manual blog posts; the ES&T (2025) research says expert validation of automated LCA is unsolved. If it were productized, neither would be true.

The one risk to hold in mind: this may be a *feature*, not a *company*. That's acceptable — its first job is to get you hired.

PART 2 — The MVP specification (the one-pager)

2.1 Scope — build ONLY this

A tool that, given (a) two versions of the DEFRA/DESNZ GHG conversion-factor workbook and (b) one product's bill-of-materials, produces a plain-English + methodology-grade report of what changed and its consequence.

Explicitly OUT of scope for the MVP (resist these — they're how the project dies):

- No ecoinvent (licensed). DEFRA only for v1.
- No multi-user accounts, no login, no cloud database, no payments.
- No fancy front-end. A single-page Streamlit app is enough.
- No coverage of every impact category — carbon (kg CO_2e) only for v1.

2.2 Data sources (all free)

Data	Use	Where
DEFRA/DESNZ GHG conversion factors — two years (e.g. 2025 + 2026)	The versioned factor tables to diff	gov.uk "Government conversion factors for company reporting"
DEFRA "major changes" report (PDF, per year)	The *reasons* the AI explains from	Same gov.uk collection
One public EPD (or a synthetic BOM CSV you write)	The stand-in "client product model"	environdec.com/library

***(optional later)* EPA Factors Hub, USEEIO**	Extend beyond UK	epa.gov

2.3 How it works (the pipeline)

```
DEFRA 2025.xlsx ┐
                ├─►[1 Normalize]─►[2 Diff]─►flagged changes ┐
DEFRA 2026.xlsx ┘                                           │
                                                            ├─►[5 AI Explain]─►Report
Product BOM (EPD/CSV)─►[3 Match to factors]─►[4 Recompute]──┘   (RAG over changes PDF)
```

5. **Normalize** — load both workbooks into one common table: `activity | unit | kgCO2e | scope | category | version`.
6. **Diff** — join the two versions on `activity+unit`, compute `% change`, flag >5% (Scope 1/2) or >10% (Scope 3) per DEFRA's own thresholds.
7. **Match** — map each BOM line item to a DEFRA activity (string match first; AI-assisted for the hard ones, with a confidence score).
8. **Recompute** — total the product footprint under both versions → absolute + % delta.
9. **AI Explain** — for each material that moved, retrieve the relevant passage from the DEFRA changes PDF and generate: (a) plain-English reason, (b) methodology-grade note, (c) a target-impact flag (e.g. "breaches a flat baseline").
10. **Output** — a Streamlit page + a downloadable Markdown/PDF report.

2.4 Tech stack (deliberately boring)

- **Python** + **pandas** (data), **Streamlit** (UI — no web skills needed), **openpyxl** (read Excel), **pdfplumber** (read the changes PDF).
- **Claude API** (the Anthropic SDK) for the explanation layer — model `claude-sonnet-5` for cost/quality balance.
- That's it. No database (use CSV/parquet files), no server, no Docker for v1.

2.5 Acceptance criteria (how you know each piece works)

- Diff produces a table where a known DEFRA change (look one up in the official changes report) appears with the correct % and flag.
- Matching maps ≥80% of a sample BOM automatically; the rest are clearly flagged "needs human match," not silently guessed.
- The AI explanation for a change **cites the DEFRA reason** and does not invent a number that isn't in the data (test this deliberately).
- The whole thing runs end-to-end on one sample product and outputs a readable report.

PART 3 — Building it with Claude Code (the core of this playbook)

3.1 What Claude Code is (plain English)

Claude Code is an AI coding assistant that runs in your terminal (or VS Code). You describe what you want in English; it writes, runs, and fixes the code, showing you each change to approve. **You don't write Python — you direct and review.** Your value is knowing what's *right* (the domain), not how to type code.

3.2 Golden rules for a non-developer using Claude Code

11. **Small steps.** Never say "build the whole app." Build one piece, test it, then the next. This playbook's prompts are already chunked this way.
12. **Make it plan first.** For anything non-trivial, ask it to *explain its plan before writing code.* Read the plan; it's in English.
13. **Always ask for a test.** Every prompt below ends by requiring a runnable check, so you can *see* it work, not take its word.
14. **Review the diff.** Claude Code shows you what changed. You don't need to understand every line — look for whether it did what you asked.
15. **When stuck, paste the error.** If something breaks, copy the whole error message back to it. That's usually all it needs.
16. **Guard the scope.** If it starts adding logins, databases, or extra features — stop it. Say "out of scope, remove it." (This is the #1 way these projects bloat and die.)

3.3 Setup

Install Claude Code (one time), then in an empty folder run it. First, give it the project's rules so every later prompt inherits context.

► Prompt 0 — Project setup & guardrails (run once, first)

```
You are helping me build a small, deliberately minimal Python tool. I am a
sustainability expert, NOT a software engineer — explain choices in plain
English and keep the code simple and readable over clever.

Create a CLAUDE.md in this folder that records these project rules, then stop
and show it to me:

PROJECT: "EF Version Explainer" — a tool that compares two annual versions of
the UK DEFRA GHG conversion-factor workbooks, flags factors that changed
beyond DEFRA's own thresholds (>5% Scope 1/2, >10% Scope 3), recalculates a
product's carbon footprint under both versions, and uses the Claude API to
explain each change in plain English + a methodology-grade note.

HARD CONSTRAINTS (do not violate without asking me):
- Stack: Python + pandas + Streamlit + openpyxl + pdfplumber + the Anthropic
  SDK only. No database, no login, no cloud, no Docker, no web framework.
- Store data as local CSV/parquet files. No servers.
- Build in small, testable steps. Never scaffold features I didn't ask for.
- Every code change must come with a way for me to SEE it work (a runnable
  check or a printed result), because I can't read code fluently.
- Carbon (kg CO2e) only for v1. No other impact categories.
- Keep functions small and named in plain language.

Set up a clean folder structure (src/, data/, tests/, and a requirements.txt)
but write NO feature code yet. Just the skeleton, CLAUDE.md, and requirements.
Then explain, in 5 bullet points, what you created and what I should do next.
```

3.4 Build phase by phase

Each prompt is copy-paste ready. Run them **in order**, and only move on once the previous one's check passes. Download the two DEFRA workbooks and the changes PDF into `data/` before Prompt 1 (Claude Code will tell you the expected filenames if you ask).

► Prompt 1 — DEFRA loader / normalizer

```
Read the plan aloud before coding.

Goal: load a DEFRA GHG conversion-factor workbook (.xlsx) and normalize it into
ONE tidy pandas table with these columns:
  activity | unit | kg_co2e | scope | category | version

The DEFRA workbook has many sheets with inconsistent headers and merged cells —
inspect the real file in data/ first and tell me what you find BEFORE writing
the parser. Handle the messiness: skip header junk, forward-fill categories,
coerce the CO2e column to numbers, drop empty rows.

Make it a function load_defra(path, version_label) -> DataFrame.

CHECK I must be able to run: a small script that loads the 2025 file, prints
the row count, the list of unique scopes, and the first 10 rows. Also assert
that kg_co2e is numeric and that there are zero fully-empty activity names.
If the assertions pass, print "LOADER OK".

Do not build anything else yet.
```

► Prompt 2 — Version diff engine

```
Read the plan aloud before coding.

Goal: given two normalized DEFRA tables (e.g. version "2025" and "2026"), join
them on (activity, unit) and produce a diff table:
  activity | unit | scope | kg_co2e_old | kg_co2e_new | pct_change | flagged

"flagged" = True when abs(pct_change) > 5% for Scope 1 or 2, or > 10% for
Scope 3 (DEFRA's own materiality thresholds). Handle activities that exist in
only one version (mark them "added" / "removed", don't crash).

Function: diff_versions(df_old, df_new) -> DataFrame.

CHECK I must be able to run: load 2025 and 2026, diff them, print how many
factors were flagged, and print the top 10 biggest movers sorted by absolute
pct_change. Then — important — I will look up ONE of those movers in the
official DEFRA changes report myself to confirm the number is real. Make the
output easy for me to eyeball for that manual check.

Handle divide-by-zero and missing values safely. Print "DIFF OK" when done.
```

► Prompt 3 — Product BOM ingestion + factor matching

```
Read the plan aloud before coding.

Goal: load a product's bill-of-materials from a simple CSV I will provide with
columns: line_item, quantity, unit. Match each line_item to the best DEFRA
activity from the normalized table.

Matching strategy, in this order (cheapest first):
1. Exact / normalized string match on activity + unit.
```

```
2. Fuzzy string match (rapidfuzz) for close names, WITH a similarity score.
3. Leave anything below a confidence threshold UNMATCHED and clearly flagged
   "needs human match" — never silently guess. A wrong match is worse than no
   match; this is a core domain rule.

Output: matched table = line_item | quantity | unit | matched_activity |
match_score | match_method | needs_review (bool).

CHECK: run on a sample BOM I'll add to data/, print how many matched
automatically vs need review, and print every match with its score so I can
audit them. Print "MATCH OK".

Do NOT call any AI model in this step — string matching only. AI-assisted
matching is a later, optional upgrade.
```

► Prompt 4 — Footprint recompute + delta

```
Read the plan aloud before coding.

Goal: using the matched BOM (from Prompt 3) and both DEFRA versions, compute
the product's total carbon footprint under the OLD and NEW factor versions:
  footprint = sum(quantity * kg_co2e) across matched line items.

Produce: a per-line table (line_item, quantity, factor_old, factor_new,
co2e_old, co2e_new, line_delta) and the product totals (total_old, total_new,
absolute_delta, pct_delta). Ignore unmatched lines but REPORT how much of the
footprint could not be computed because of unmatched items (a coverage %).

Function: recompute(matched_df, diff_df) -> (line_table, summary_dict).

CHECK: run end-to-end on the sample product, print the summary (old total, new
total, % change, and coverage %), and the 5 line items contributing most to the
delta. Print "RECOMPUTE OK".
```

► Prompt 5 — The AI explanation layer (the actual moat)

```
Read the plan aloud before coding. This is the most important step — go slowly.

Goal: for each FLAGGED, footprint-relevant change, use the Claude API
(model claude-sonnet-5) to generate an explanation grounded in the official
DEFRA "major changes" report PDF in data/.

Pipeline:
1. Use pdfplumber to extract the changes-report text; chunk it and keep it
   searchable (simple keyword/embedding retrieval is fine — no vector DB, keep
   it in-memory).
2. For each flagged material that appears in the product's delta, retrieve the
   relevant passage(s) from the report.
3. Call Claude with a STRICT prompt (see below) to produce, per material:
   - plain_english_reason (2-3 sentences, non-expert readable)
   - methodology_note (1-2 sentences, standards-aware, GHGP/ISO tone)
   - target_impact_flag (e.g. "would breach a flat baseline" / "immaterial")

CRITICAL GROUNDING RULES for the Claude call — bake these into the system prompt:
- Only use numbers I pass in the input. NEVER invent or estimate a factor.
```

```
- If the retrieved report text does not explain a change, say exactly
  "No official reason found in the DEFRA changes report" — do not speculate.
- Keep methodology claims consistent with GHG Protocol / ISO 14064 framing.
- Output strict JSON with the three fields above, nothing else.

Build a function explain_change(material, old, new, pct, retrieved_text) -> dict.

CHECK: run it on ONE flagged material and print the JSON. Then run a
DELIBERATE trap test: pass a material whose reason is NOT in the report text
and confirm the model returns "No official reason found..." instead of making
something up. Print "EXPLAIN OK" only if the trap test passes.
```

► Prompt 6 — Report + Streamlit dashboard

```
Read the plan aloud before coding.

Goal: wire steps 1-5 into a single Streamlit page. Layout:
- Sidebar: pick the two DEFRA versions and upload a product BOM CSV.
- Main: a headline "Footprint moved from X to Y (+Z%)", a coverage % note, a
  sortable table of the biggest movers, and an expandable section per flagged
  material showing the plain-English reason, methodology note, and target flag.
- A "Download report" button that exports the same content as a Markdown file
  (PDF export optional, only if trivial).

Keep it one file (app.py). No auth, no themes, no extras.

CHECK: give me the exact command to launch it, and a screenshot-free checklist
of what I should see on screen for the sample product. Print "APP OK".
```

► Prompt 7 — Sample data + one-command demo

```
Create a realistic but SYNTHETIC sample product BOM (based on a public EPD I'll
name, or invent a plausible consumer product) and save it to data/sample_bom.csv.
Then create a single script run_demo.py that executes the whole pipeline start
to finish on the sample and DEFRA 2025 vs 2026, printing each stage's summary,
so I can prove the whole thing works with one command.

CHECK: tell me the one command to run, and what the final output should say.
```

3.5 After it works — optional upgrades (only if the demo lands)

Do these **only** if someone (an employer, a design partner) shows interest. Otherwise stop — a working demo is the deliverable.

- AI-assisted matching for the "needs human match" leftovers (extend Prompt 3).
- Add EPA / USEEIO datasets (extend Prompt 1's loader).
- Batch mode: run a whole portfolio of products, not one.
- Export a client-ready PDF with your branding.

PART 4 — How to frame this on your job applications

Attach or mention it on the roles where it's a bullseye. Suggested one-liner for a cover letter or CV "Projects" line:

> **Built an AI tool (Python + Claude) that compares annual emission-factor database versions, recomputes product footprints under each, and generates methodology-grade explanations of what changed and why — automating the manual EF-reconciliation and trend-reporting work I did at Quantis.**

Per-target angle:

- **carbmee / Makersite / CO2 AI-adjacent:** lead with the *version-reconciliation + explanation* engine — it's their world.
- **Planet A / Footprint Intelligence:** lead with *turning free public data into a client-ready methodology narrative.*
- **Henkel "Digital Data Architect":** lead with the *data normalization + governance + provenance* angle (mirrors your Product Owner CFDB work).
- **Consultancy roles (Rödl, Horváth, Simon-Kucher):** lead with *"packaged the expert judgment that clients currently pay day-rates for."*

PART 5 — What NOT to do (so the project survives)

17. **Don't buy ecoinvent.** Not until you have income or an employer's licence.
18. **Don't add a database / login / payments.** Files on disk are fine for a demo.
19. **Don't try to cover every impact category or every dataset.** Carbon + DEFRA proves the concept.
20. **Don't let the AI silently guess an EF match or invent a reason.** The whole credibility of the tool — and of *you* — is that it flags uncertainty instead of faking confidence. This is the domain rule generic tools get wrong; getting it right is your differentiation.
21. **Don't polish before it works end-to-end.** Ugly-but-working beats pretty-but-broken for a portfolio piece.
22. **Don't build in secret for months.** Get the rough demo in front of 2-3 target employers early and let their reaction tell you whether to go further.

Next actions: (1) download DEFRA 2025 + 2026 workbooks + the changes report; (2) install Claude Code; (3) run Prompt 0. Everything after that is directed conversation.